Agenda

- List
- Queue
- ~~Set~~
- ~~Map~~
- ~~hashcode()~~

List Interface

- Ordered/sequential collection.
- Implementations: ArrayList, Vector, Stack, LinkedList, etc.
- List can contain duplicate elements.
- List can contain multiple null elements.
- Elements can be accessed sequentially (bi-directional using Iterator) or randomly (index based).
- List enables searching in the list
- Abstract methods
  - void add(int index, E element)
  - String toString()
  - E get(int index)
  - E set(int index, E element)
  - int indexOf(Object o)
  - int lastIndexOf(Object o)
  - E remove(int index)
  - boolean addAll(int index, Collection<? extends E> c)
  - ListIterator listIterator()
  - ListIterator listIterator(int index)
  - List subList(int fromIndex, int toIndex)
- To store objects of user-defined types in the list, you must override equals() method for the objects.
- It is mandetory while searching operations like contains(), indexOf(), lastIndexOf().

```
// List<String> l1 = new ArrayList<>(); // OK
// List<String> l1 = new Vector<>();    // OK
List<String> l1 = new LinkedList<>();
```

- List interface provides the index based operations

```
List<String> l1 = new ArrayList<>();
l1.add("India");
l1.add("USA");
l1.add("UK");

l1.add(1, "China"); // add element at specified index
String country = l1.get(2); // get element from specified index

l1.set(3, "Japan"); // change the value at the given index
```

SUNBEAM INFOTECH

```
l1.remove(2); // remove the element from the specified index

int index = l1.indexOf("Japan"); // serach the element and returns its index, if not found returns -1
```

ArrayList class

- Internally ArrayList is dynamic array (can grow or shrink dynamically).
- When ArrayList capacity is full, it grows by half of its size.
- Elements can be traversed using Iterator, ListIterator, or using index.
- Primary use
  - Random access
  - Add/remove elements (at the end)
- Limitations
  - Slower add/remove in between the collection
  - Uses more contiguous memory
- Inherited from List<>.

Vector class

- Internally Vector is dynamic array (can grow or shrink dynamically).
- Vector is a legacy collection (since Java 1.0) that is modified to fit List interface.
- Vector is synchronized (thread-safe) and hence slower.
- When Vector capacity is full, it doubles its size.
- Elements can be traversed using Enumeration, Iterator, ListIterator, or using index.
- Primary use
  - Random access
  - Add/remove elements (at the end)
- Limitations
  - Slower add/remove in between the collection
  - Uses more contiguous memory
  - Synchronization slow down performance in single threaded environment
- Inherited from List<>.

LinkedList class

- Internally LinkedList is doubly linked list.
- Elements can be traversed using Iterator, ListIterator, or using index.
- Primary use
  - Add/remove elements (anywhere)
  - Less contiguous memory available
- Limitations:
  - Slower random access
- Inherited from List<>, Deque<>.

Stack

SUNBEAM INFOTECH

- It is inherited from vector class.
- Generally used to have only the stack operations like push, pop and peek opertaions.
- It is recommended to use the Dequeu from the queue collection.
- It is synchronized and hence gives low performanance.

Queue Interface

- Represents utility data structures (like Stack, Queue, ...) data structure.
- Implementations: LinkedList, ArrayDeque, PriorityQueue.
- Can be accessed using iterator, but no random access.
- Methods
  - boolean add(E e) - throw IllegalStateException if full.
  - E remove() - throw NoSuchElementException if empty
  - E element() - throw NoSuchElementException if empty
  - boolean offer(E e) - return false if full.
  - E poll() - returns null if empty
  - E peek() - returns null if empty
- In queue, addition and deletion is done from the different ends (rear and front).
- Difference between these methods is first 3 methods throws exception however next 3 methods do not throw exception if operation fails.

```
// Queue<String> q = new PriorityQueue<>(); // OK
// Queue<String> q = new LinkedList<>(); // OK
Queue<String> q = new ArrayDeque<>();
q.offer("One");
q.offer("Two");
System.out.println("First Element - " + q.peek());
// If operation fails then it return null
System.out.println("Popped element : " + q.poll()); // null

q.add("Three");
q.add("Four");
System.out.println("First Element - " + q.element());
// If operation fails then it throws exception
System.out.println("Popped element : " + q.remove()); // NoSuch Elementexception
```

Deque interface

- Represents double ended queue data structure i.e. add/delete can be done from both the ends.
- Two sets of methods
  - Throwing exception on failure: addFirst(), addLast(), removeFirst(), removeLast(), getFirst(), getLast().
  - Returning special value on failure: offerFirst(), offerLast(), pollFirst(), pollLast(), peekFirst(), peekLast().
- Can used as Queue as well as Stack.
- Methods
  - boolean offerFirst(E e)

SUNBEAM INFOTECH

- E pollFirst()
- E peekFirst()
- boolean offerLast(E e)
- E pollLast()
- E peekLast()

```
// Deque<Integer> q = new LinkedList<>(); // OK
Deque<Integer> q = new ArrayDeque<>();
q.offerFirst(10);
q.offerFirst(20);

q.offerLast(30);
q.offerLast(40);

// If operation fails then it return null
System.out.println("Popped element : " + q.pollFirst());
System.out.println("Popped element : " + q.pollLast());
```

ArrayDeque class

- Internally ArrayDeque is dynamically growable array.
- Elements are allocated contiguously in memory.
- Time Complexity to add and remove is O(1)

LinkedList class

- Internally LinkedList is doubly linked list.
- Time Complexity to add and remove is O(1)

PriorityQueue class

- Internally PriorityQueue is a "binary heap" (Array implementation of binary Tree) data structure.
- Elements with highest priority is deleted first (NOT FIFO).
- Elements should have natural ordering or need to provide comparator.

```
// elements are reterived as per the priority
// decided by the Comparable (Natural Ordering)
Queue<String> q = new PriorityQueue<>();
q.offer("I");
q.offer("N");
q.offer("F");
q.offer("O");
q.offer("T");
q.offer("E");
q.offer("C");
q.offer("H");

while(!q.isEmpty()) {
```

SUNBEAM INFOTECH

```
        String ele = q.poll();
        System.out.print(ele + ",");
    }
```

SUNBEAM INFOTECH